UNICESUMAR

Aula 013

Aluno: Matheus Costa e Silva — **RA:** 24000729-2

Maringá

19/10/2026

1. Unificação

- **`?- f(X, b) = f(a, Y).`**
  - **(a) Resposta:** `X = a, Y = b.`
  - **(b) Regra:** O operador `=` realiza unificação sintática. Termos com o mesmo functor e aridade unificam se seus argumentos correspondentes unificarem. Assim, `X` se associa a `a` e `Y` se associa a `b`.
- **`?- f(X, X) = f(a, b).`**
  - **(a) Resposta:** `false.`
  - **(b) Regra:** A unificação exige consistência para a mesma variável. O primeiro argumento define `X = a`. O segundo argumento exige `X = b`. Como `X` não pode ser ambos, a unificação falha.

---

2. Aritmética

- **`?- X = 2 + 3.`**
  - **(a) Resposta:** `X = 2 + 3.`
  - **(b) Regra:** O operador `=` não avalia operações matemáticas. Ele realiza apenas casamento de padrões puramente estruturais. A variável `X` unifica diretamente com o termo composto `+(2, 3)`.
- **`?- X is 2 + 3.`**
  - **(a) Resposta:** `X = 5.`
  - **(b) Regra:** O operador `is` força a avaliação aritmética do lado direito. O resultado da soma (`5`) é então unificado com a variável `X`.
- **`?- 5 = 2 + 3.`**
  - **(a) Resposta:** `false.`
  - **(b) Regra:** O operador `=` compara as estruturas sintáticas. O número `5` é estruturalmente diferente do termo composto `2 + 3`. Nenhuma conta é realizada, resultando em falha.

Exercício 3 · Ordem das respostas

Código de base:

prolog

```
cor(maca, vermelha).
cor(banana, amarela).
cor(morango, vermelha).

?- cor(F, vermelha).
```

- **(a) O que o Prolog responde?**
  O Prolog busca por fatos que correspondam à consulta de cima para baixo. Como existem duas soluções, se você pedir por mais respostas (usando `;`), ele retornará:
  1. `F = maca`
  2. `F = morango`
- **(b) Qual regra explica?**
  Regra da Ordem de Busca (Top-Down): O Prolog avalia a base de dados sequencialmente, do primeiro ao último fato declarado. Ele encontra primeiro `cor(maca, vermelha)` e depois `cor(morango, vermelha)`, ignorando `cor(banana, amarela)` porque o segundo argumento não unifica com `vermelha`.

Exercício 4 · Listas

Item 1: `?- [H | T] = [a, b, c].`

- **(a) O que o Prolog responde?**
  - `H = a`
  - `T = [b, c]`
- **(b) Qual regra explica?**
  Regra do Operador de Cabeça e Cauda (`|`): O operador divide uma lista não vazia em duas partes: a **Cabeça (`H`)**, que é o primeiro elemento individual, e a **Cauda (`T`)**, que é uma lista contendo todos os elementos restantes.

Item 2: `?- [A, B | R] = [1, 2].`

- **(a) O que o Prolog responde?**
  - `A = 1`
  - `B = 2`
  - `R = []`
- **(b) Qual regra explica?**
  Regra de Unificação com Lista Vazia: O Prolog extrai o primeiro elemento para `A` e o segundo para `B`. Como a lista original só possui dois elementos, não sobra nenhum item para o resto (`R`). Portanto, `R` unifica com a **lista vazia (`[]`)**.

Exercício 5: Negação

A base de conhecimento possui apenas o fato: `gosta(ana, prolog).`

O operador `\+` representa a **Negação por Falha** (*Negation as Failure*). Ele retorna `true` se o Prolog **não conseguir provar** o objetivo, e `false` se conseguir.

Primeira consulta: **`?- \+ gosta(bruno, prolog).`**

- **(a) O que o Prolog responde:** `true.`
- **(b) Qual regra explica:** O Prolog tenta provar `gosta(bruno, prolog)`. Como esse fato não está na base de dados (e nenhuma regra o gera), a busca falha. Pela negação por falha, como o objetivo falhou, o operador `\+` inverte o resultado e retorna sucesso (`true`).

Segunda consulta: **`?- \+ gosta(X, prolog).`**

- **(a) O que o Prolog responde:** `false.`
- **(b) Qual regra explica:** O Prolog tenta provar `gosta(X, prolog)`. Ele encontra uma unificação válida substituindo `X = ana`. Como o objetivo interno foi provado com sucesso (não falhou), o operador de negação `\+` inverte o resultado, fazendo a consulta inteira falhar (`false`).

Exercício 6: Ordem das cláusulas

Consulta: **`?- ancestral(ana, bia).`**

- **(a) O que o Prolog responde:** O programa entra em um **loop infinito** (estouro de pilha / *stack overflow*).

- **(b) Qual regra explica:** O Prolog avalia as cláusulas de cima para baixo e da esquerda para a direita.